

<https://docs.google.com/spreadsheets/d/1oMYa18daODYem6uJOupDJyr5VOri1WtY2ALIBMgBcr0/edit?gid=0#gid=0>

Application functionality.

Our project's core use case, Scheduled Automated Data Synchronization, is implemented and working. The system uses a scheduler to automatically trigger jobs for worker that discovers infrastructure assets with Ansible and then synchronizes that data with our Jira CMDB. Also, the conditions for UC-02 are implemented.

To run our project you need to have Linux machine/environment. To test it, run main.py file and test it via localhost:8000/docs.

Siia vaja screene töötavast rakendusest. Ehk ss terminalidest ja jira poolest meie juhul. Samuti vaja mingid Use case TESTID teha ja märkida passed/failed.

CMDB API 0.1.0 OAS 3.1	
/openapi.json	
Jira Integration ^	
GET	/jira/assets Get Jira Assets v
GET	/jira/schemas Get Jira Schemas v
Jira Integration (Debug) ^	
GET	/jira/schemas Get Jira Schemas v
Assets ^	
GET	/assets/ Get All Assets v
Jobs ^	
GET	/jobs/ Get All Jobs v
GET	/jobs/{task_id} Get Job Status v
GET	/jobs/active Get Active Jobs v
POST	/jobs/{task_id}/cancel Cancel Job v
GET	/jobs/stats Get Job Stats v
Sync ^	

For the last release, we plan to have a .bat file for running the whole project.

Continuous Integration (CI).

Continuous Integration is used throughout the process. Everyone has their own branch on Github and we are continuously committing and merging branches. Since we do not deploy our project somewhere, we do not use continuous deployment.

Detailed requirements. At least 75% of the requirements identified and marked as in the scope of the work delivered at the end of the course are detailed in the form of use cases). Detailed requirements are in individual wiki pages/documents. Detailed requirements are accessible from the overall requirements list. For any UML artifacts, you may also use an external UML tool and add links in your Wiki to these artifacts or just upload them to your Wiki.

Terminali screenid lisada wikisse.

UI wireframes/mockups. If applicable, the application UI has been prototyped using wireframes or mockups and there are links from the detailed requirements to the corresponding UI screens. In the case of a non-interactive application (such as a command-line app), there should be representative examples of "input-output" pairs (instead of UI mockups), and these examples should be linked to the requirements.

Assets / Schemas

Test Ansible

SCHEMA TREE

Hardware Assets

Phones

Laptops

Servers

Ubuntu

Window...

Printers

Models

Hardware Mo...

Model Categ...

Software Mo...

1 object

Tower of Chronia Test

TA-2

Tower of Chronia Test

Details

Activity

Name	Tower of Chronia Test
CPU Temperature	35.0
CPU Model	test
Memory (GB)	43.0
Disk Usage	24.0
IP Address	test
Hostname	Test
Architecture	Test
Processor Model	None entered
Physical CPUs	None entered
Virtual CPUs	None entered

Scheduler work:

```

● philippaskov@Philips-MacBook-Pro UT_CMDB % source venv/bin/activate
○ (venv) philippaskov@Philips-MacBook-Pro UT_CMDB % celery -A src.scheduler.celery_app beat --loglevel=info
Configuration file not found: /Users/philippaskov/Kool/semester5/softwareproject/UT_CMDB/src/core/config/conf
ig.json
Configuration file not found: /Users/philippaskov/Kool/semester5/softwareproject/UT_CMDB/src/core/config/conf
ig.json
celery beat v5.5.3 (immunity) is starting.
_ _ _ _ _
LocalTime -> 2025-10-19 20:49:17
Configuration ->
  . broker -> sqlalchemy+sqlite:///celery_broker.db
  . loader -> celery.loaders.app.AppLoader
  . scheduler -> celery.beat.PersistentScheduler
  . db -> celerybeat-schedule
  . logfile -> [stderr]@%INFO
  . maxinterval -> 5.00 minutes (300s)
[2025-10-19 20:49:17,753: INFO/MainProcess] beat: Starting...
[2025-10-19 20:49:17,775: INFO/MainProcess] Scheduler: Sending due task periodic-ansible-discovery (worker.ta
sks.discovery.discovery_task)
[2025-10-19 20:49:17,784: INFO/MainProcess] Scheduler: Sending due task periodic-linux-discovery (worker.task
s.discovery.discovery_by_type_task)
[2025-10-19 20:49:17,785: INFO/MainProcess] Scheduler: Sending due task periodic-auto-discovery (worker.tasks
.discovery.auto_discovery_task)

```

Worker work:

```
o (venv) philippaskov@Philips-MacBook-Pro UT_CMDB % celery -A src.worker.main worker --loglevel=info

----- celery@Philips-MacBook-Pro.local v5.5.3 (immunity)
--- ***** ---
-- ***** --- macOS-15.6.1-arm64-arm-64bit 2025-10-19 20:50:13
- *** --- * ---
- ** ----- [config]
- ** ----- .> app: cmdb_worker:0x107ebd9d0
- ** ----- .> transport: sqlalchemy+sqlite:///celery_broker.db
- ** ----- .> results: sqlite:///celery_results.db
- *** --- * --- .> concurrency: 2 (prefork)
-- ***** --- .> task events: OFF (enable -E to monitor tasks in this worker)
--- ***** ---

----- [queues]
-> ansible exchange=ansible(direct) key=ansible

[tasks]
. worker.tasks.discovery.auto_discovery_task
. worker.tasks.discovery.batch_discovery_task
. worker.tasks.discovery.discovery_by_type_task
. worker.tasks.discovery.discovery_task
. worker.tasks.sync_to_jira.sync_discovered_assets
. worker.tasks.sync_to_jira.sync_task

[2025-10-19 20:50:13,995: INFO/MainProcess] Connected to sqlalchemy+sqlite:///celery_broker.db
[2025-10-19 20:50:14,001: INFO/MainProcess] celery@Philips-MacBook-Pro.local ready.
```

Scope

- **Iteration 3:**

- More functionality
- Add support for Windows and Solaris systems
- Implement asset change tracking and audit logs
- Expand test coverage and validation logic

- **Iteration 4:**

- Finalization and Delivery
- Optimize performance and security
- Finalize documentation and architectural diagrams
- Conduct user testing and gather feedback
- Prepare deployment package and handover materials

Release notes

Available in Github.

Collaboration infrastructure is in use.

VCS is being used regularly and by all team members. Requirements in wiki or other documents are updated regularly and the issue tracking system is also updated and used regularly.

Individual contribution report

Madis - worker with core/ folder and overall logic of the project, connected different parts together

Philip - built Scheduler and its connections to other parts

Kristjan - built API part and worked with Jira

Oskar - built Worker and its connections to other parts